

Group Project: Celebrity Name Chain

Assigned: 2026-07-01 · **Due:** *TBD* · **Team size:** 2–3

Purpose

- To practice **Ionic**, **React Hook Form**, and **TanStack Query** by building a **multi-user game**.
- To build a **full-stack** app: an **Express + Prisma + PostgreSQL** API with an **Ionic React** client.
- To practice **live updates** with **polling** (TanStack Query's `refetchInterval`).
- To reinforce **server-enforced rules** and validating untrusted **user input** on the server.
- To have fun.

The Game

Based on the **Celebrity Name Chain** game: players name a celebrity (first and last name), and each new celebrity's **first name** must start with the **first letter of the previous celebrity's last name**.

- “Albert **E**instein” → “Elvis Presley” → “Pablo Picasso” → ...
- There is **no turn order** — any player can shout an answer at any time
- The one rule on *who* may answer: the player who submitted the **most recent** celebrity **cannot answer again in a row**; their answer is rejected
- Players join with a **room code** and a **username** (no accounts, no auth)
- The server is the **referee**: it decides whether a name legally continues the chain

Task

Build a **full-stack** app in two parts:

- **Backend:** an **Express API** (Prisma + PostgreSQL)
- **Frontend:** an **Ionic React** client (React Hook Form + TanStack Query)

API routes (at least)

- **New game:** `POST /games` with `{ roomCode, celebrity }`. The `celebrity` is a single string used as the **starting name** and is **always accepted**. Creates the game and returns it.
- **List games:** `GET /games`. Returns every game with just its **room code** and **most recent** celebrity name (no player counts or presence).
- **Get game:** `GET /games/:roomCode`. Returns the **most recent** celebrity name (nothing else is required).
- **Submit answer:** `POST /answers` with `{ roomCode, username, answer }`, where `answer` is a **single string**. The **server** splits it on spaces to determine the first and last name.

Rules the server enforces

- **Room code uniqueness** is checked **only when starting a new game**: if a game with that `roomCode` already exists, reject the new game (e.g. `409`). Submitting answers never checks this.
- For **every accepted answer**, the answer's **first name** must **start with the first letter of the previous celebrity's last name**, or respond with an error (e.g. `400`).
- The player who submitted the **most recent** celebrity **cannot answer again in a row**: reject it (e.g. `409`).
- **No auth** (to keep things simple): the **username** is only a label, and we trust players to use their own name.

Relationship (required)

- A **game** has many **answers** (`Game` 1:N `Answer`).

The Client (Ionic + React Hook Form + TanStack Query)

Build an **Ionic React** UI that plays the game:

- Show only the **most recent** celebrity name (not the whole chain, and no “next letter” hint)
- Three `IonInput`s — **room code**, **username**, and **answer** — managed with **React Hook Form**; on submit, `reset()` **only the answer** field (keep room code and username)
- Load the game with `useQuery`, and **poll** with `refetchInterval` so the latest name updates **live**

- Submit with a `useMutation`; on success, **invalidate** (or `refetch`) the game so the newest name shows up
- Pick a sensible polling interval, and be ready to explain the **freshness vs. efficiency** trade-off

Exposing Your Server

So classmates can play together, expose your Express API with **ngrok**:

```
ngrok http 3000
```

ngrok creates a temporary **public URL** that tunnels requests to a port on your machine (here, your API on `localhost:3000`). Point every player's client at that URL. Expose the **API**, never your database directly.

Example Exchanges

```
POST /games { "roomCode": "STAR01", "celebrity": "Albert Einstein" }
{ "roomCode": "STAR01", "mostRecent": "Albert Einstein" }
```

```
POST /games { "roomCode": "STAR01", "celebrity": "Marie Curie" }
409 { "error": "Room code already in use" }
```

```
GET /games
[
  { "roomCode": "STAR01", "mostRecent": "Albert Einstein" },
  { "roomCode": "KIWI88", "mostRecent": "Serena Williams" }
]
```

```
GET /games/STAR01
{ "roomCode": "STAR01", "mostRecent": "Albert Einstein" }
```

```
POST /answers { "roomCode": "STAR01", "username": "kai", "answer": "Elvis Presley" }
{ "accepted": true, "mostRecent": "Elvis Presley" }
```

```
POST /answers { "roomCode": "STAR01", "username": "kai", "answer": "Pablo Picasso" }
409 { "error": "You answered most recently; wait for someone else" }
```

```
POST /answers { "roomCode": "STAR01", "username": "sam", "answer": "Tom Hanks" }
400 { "error": "Name must start with P" }
```

```
POST /answers { "roomCode": "STAR01", "username": "sam", "answer": "Pablo Picasso" }
{ "accepted": true, "mostRecent": "Pablo Picasso" }
```

Sharing the Database

Your teammates need the same tables. Two ways to move a database between machines:

- **A committed SQL dump (recommended).** Export your database to a plain `dump.sql` (for example with `pg_dump`) and **commit it to your repo's `data/` folder**. It's the easiest way to share a starting point between teammates, and it makes the final submission easy to review.
- **pgAdmin backup/restore.** Right-click your database → **Backup...** to save a dump; on another machine, create an empty database → **Restore...** and select that file. Official guide: https://www.pgadmin.org/docs/pgadmin4/latest/backup_and_restore.html

Whichever you use, everyone should restore from the **same** dump so you all start from identical data.

Repo Structure

Keep the project organized, for example:

```
celebrity-name-chain/
├─ api/          # Express + Prisma server (the game API)
├─ client/       # Ionic React app (RHF + TanStack Query)
├─ data/         # database dump to import (dump.sql)
└─ README.md    # how to import the data and run everything
```

Your `README` should clearly explain:

- How to **import the database** (restore the dump in `data/`)
- How to set up `.env` (your `DATABASE_URL`)
- How to **run the API** and the **client**, and how to expose the API with ngrok

Starter Repo (Use This Template)

You do **not** start from scratch. I will provide a boilerplate repo with `api/` and `client/` already scaffolded:

<https://github.com/jonathan-chin/citytech-ttpr-2026-summer-celebrity-name-chain>

- Click **"Use this template"**, **not Fork**.
 - **Use this template** creates a fresh, independent repo with clean history and no link back to mine — right for starting your own project.
 - **Fork** keeps a permanent link to the upstream repo and is meant for contributing changes back, which is not what we want here.
- **Only one team member** creates the repo from the template, then **adds the others as collaborators**.
- **One team = one repo**.

Requirements and Allowances

- Tracked in a **GitHub repo from the start**, with **regular commits from all members**.
- Backend uses **Prisma + PostgreSQL**; the `Game 1:N Answer` relationship is required.
- Client uses **Ionic, React Hook Form, and TanStack Query** (with polling).
- You may use **AI** at any stage, but **disclose** how and where you used it.
- You may use **third-party packages**; note which ones and why.

Deliverables

- A link to your **GitHub repository**.
- A working **Express + Prisma API** that runs the game and enforces the rules.
- An **Ionic React client** that plays the game and polls for live updates.
- A short **README**: how to run the API and client, how to expose it with ngrok, and your AI-use disclosure.

Submission

- Push your final work to GitHub.
- Submit the repo link via the **repo submission form** in slack.

Tiered Learning Goals

- **Normalize answers:** store names in a consistent form (lowercase, trimmed) so “Elvis Presley” and “ elvis presley ” count as the same.
- **Validate real celebrities:** check each answer against the **free Wikipedia search API** (no API key; just send a descriptive `User-Agent`), e.g. `https://en.wikipedia.org/w/api.php?action=opensearch&search=<name>` , and reject names that don’t match a real article.
- **Answer history:** expand `GET /games/:roomCode` to return **every** answer, and show an **auto-scrolling, intuitive history** list in the client (not just the most recent name).
- **No repeats:** prevent the same celebrity being used twice in a game with a **UNIQUE** constraint on `(game_id, normalized_name)` , and have Express catch the error and reject the duplicate.
- **Guard the room code in the database:** add a **UNIQUE** constraint on `room_code` so two games can never share a code, even under a race.
- **Scoreboard:** add a `scores` table that tracks each player’s accepted answers across games.
- **Round timer:** stamp each answer with a timestamp and reject answers that arrive after a time limit.
- **Optimistic updates:** show a submitted answer **immediately** with TanStack Query’s optimistic updates, rolling back if the server rejects it.
- **Swap polling for WebSockets:** replace `refetchInterval` with a real-time socket connection and compare the two.
- **Smart name parsing:** handle middle names, suffixes (“Jr.”), and single-word names when splitting the answer into first and last name.
- **Deploy** the API so it is online without ngrok.
- **Auth:** add real player accounts and logins.